

Advanced Compilers at the Orthogonal School

Welcome to the my one-week introduction to the wonders of language implementation! The materials this week are based on the first few lessons of [CS 6120](#), so check out those materials for more details or ways to continue the fun after this week.

The main purpose of this page is to remind you of the assigned tasks for each lecture. I'll keep it updated throughout [the BOOST week](#); the schedule may shift depending on how quickly we're able to move.

Monday: Representing Programs

We'll start by thinking about what *language implementation* encompasses and then survey the different ways that implementations can represent programs. I'll also introduce [Bril](#), the instruction-based program representation we'll use this week.

Your Task

- Set up the [Bril](#) tools.
 - Clone [the repository](#).
 - Install [Deno](#) and [uv](#).
 - Install the reference interpreter by typing `deno install -g brili.ts`. As Deno instructs, add `$HOME/.deno/bin` to your `$PATH`.
 - Install the Python text-format tools using [uv](#) by typing `uv tool install .` in the `bril-txt` directory.
- For a walkthrough of the steps for getting started with these tools, see [this video introduction to Bril](#). Watch the first few minutes but skip the outdated installation steps

name index.md
last edit 2026-07-28
by Adrian Sampson
commit cf43e36

Advanced Compilers at the Orthogonal School

Monday: Representing Programs

Your Task

Tuesday Morning: Local Analysis & Optimization

LVN Pseudocode

Your Task

Tuesday Afternoon: Data Flow

Worklist Algorithm Pseudocode

Your Task

Wednesday: Global Analysis

Your Task

Thursday: Static Single Assignment

Your Task

that start around 5:20 (we now use `deno` and `uv` instead of `yarn` and `flit`). More useful instructions start again around 7:30.

- Implement the algorithms to form basic blocks and build a control flow graph.
 - Implement some tests for your analysis tool. One option is to use [Turnt](#).
- **Optional:** Write a new benchmark!
 - You can write it by hand, use the [TypeScript compiler](#), or generate it some other way.
 - Try running it with [brili](#).
 - Open a pull request to add your new benchmark to the repository!
 - Add your code to the [the benchmarks directory](#).
 - Use `turnt --save yours.bril` to create the test outputs for your new benchmark. (See the [Turnt](#) README for details.)
 - If your `@main` function takes arguments, you can specify ones to use in testing with an `ARGS:` comment, [like this](#).
 - Mention it in [the docs](#).

Tuesday Morning: Local Analysis & Optimization

We'll introduce our first optimizations: dead code elimination (as a warm-up) and local value numbering (the main event). I'll demonstrate some approaches to testing.

LVN Pseudocode

Here's the pseudocode from the slide:

```
table = mapping from value tuples to canonical variables,
        with each row numbered
var2num = mapping from variable names to their current
        value numbers (i.e., rows in table)

for instr in block:
    value = (instr.op, var2num[instr.args[0]], ...)

    if value in table:
        # The value has been computed before; reuse it.
```

```

num, var = table[value]
replace instr with copy of var

else:
    # A newly computed value.
    num = fresh value number

    dest = instr.dest
    if instr will be overwritten later:
        dest = fresh variable name
        instr.dest = dest
    else:
        dest = instr.dest

    table[value] = num, dest

    for a in instr.args:
        replace a with table[var2num[a]].var

var2num[instr.dest] = num

```

Your Task

- Implement “trivial” dead code elimination, in which you delete instructions that are never used before they are reassigned. Remember to iterate to convergence. (This should not take you long.)
- Implement local value numbering. Make sure it eliminates some common subexpressions. Try pairing it with trivial dead code elimination as a post-processing step. (This will take longer.)
- Test your implementation.
 - The key thing you’ll want to test is that your optimizations don’t break programs: i.e., that some Bril programs “do the same thing” before and after applying your optimization. Consider using [all the “core” benchmarks in the Bril repository](#), for example.
 - Secondly, you can also consider measuring the efficiency effects of your optimization: i.e., do your optimizations actually make programs faster? You can measure the number of instructions that a program executes by using `brili -p` (the `-p` flag is for “profile”).

- One option is to use [Brench](#), but this is totally open-ended.
- For bonus “points,” extend your LVN implementation to optimize the trickier examples given in class.

Tuesday Afternoon: Data Flow

The *data flow* framework is a powerful and general mechanism for analyzing entire functions in a control-flow-sensitive way. You can think of it as a way to take local analyses and give them global superpowers.

For more on the theory, I recommend Chapter 5 of [Static Program Analysis by Møller and Schwartzbach](#).

Worklist Algorithm Pseudocode

Here’s the pseudocode for solving a forward data flow problem with a worklist algorithm:

```
in[entry] = init
out[*] = init

worklist = all blocks
while worklist is not empty:
    b = pick any block from worklist
    in[b] = merge(out[p] for every predecessor p of b)
    out[b] = transfer(b, in[b])
    if out[b] changed:
        worklist += successors of b
```

For the backward version, flip predecessors & successors, and flip *in* and *out*.

Your Task

- Implement at least one data flow analysis. You choose which.
- For bonus “points,” implement a generic solver that supports multiple analyses.
- As always: Try to convince yourself that your analyses are working how you want them to (i.e., set up some form of testing). Because this is just an analysis and not an

optimization, you will have to get creative with your testing setup.

Wednesday: Global Analysis

We'll talk about control flow graphs (CFGs) and define many things we want to deduce about them.

Your Task

- Implement some dominance utilities:
 - Find dominators for a function.
 - Construct the dominance tree.
 - Compute the dominance frontier.
- Devise a way to test your implementations.
 - One way is to simply add some snapshot tests for handcrafted tests.
 - A more ambitious option is to think of a “test oracle” for our algorithms. For example, is there a way to algorithmically confirm that a block *A* dominates a block *B*? While *computing* these sets should be cheap, *checking* their output could use slow, naive algorithms.

Thursday: Static Single Assignment

Static single assignment (SSA) form is an incredibly popular program representation that makes code easier to reason about and analyses easier to write. We'll talk about how SSA works and some algorithms for converting a standard program to and from SSA form.

Your Task

- Implement the “into SSA” and “out of SSA” transformations on Brill functions.
 - For both directions, you can pick which version of the transformation to implement. For the “into SSA” transformation, for example, you can try the [basic](#)

- [version](#) (which should be pretty straightforward), the [dominance-frontier-based version](#) (which is a bit harder), or any other technique you find in the literature.
- One thing to watch out for: a tricky part of the translation from the pseudocode to the real world is dealing with variables that are undefined along some paths. The [Bril SSA extension](#) has an `undef` instruction you can use to handle this case.
- Previous 6120 adventurers have found that the “full” version of the “into SSA” transformation can be tricky to get right. Leave yourself plenty of time, and test thoroughly.
- Convince yourself that your implementation actually works!
 - Check that the output of your “to SSA” pass is *actually* in SSA form. There’s a [really simple is_ssa.py script](#) that can check that for you.
 - You also must check that programs do the same thing when converted to SSA form and back again. Fortunately, [brili](#) supports the `phi` instruction, so you can interpret your SSA-form programs if you want to check the midpoint of that round trip.
- **Optional:** Measure the overhead. If you take a program on a round trip through SSA form and back, how many more instructions (static or dynamic) does the final program have than the original? Report this overhead so you can compare your implementation against the rest of the class. If you ended up implementing more than one version of the transformations, you can compare them against each other.